

11.2 Using OpenHands

The first task for our artificial junior software developer is to create a web application with very simple user administration. The backend is to be programmed in Python, and the users are to be stored in an SQLite database. A website is used to manage users, where you can add new users and delete or change existing ones.

Prompt: Create a JSON REST API backend using Python. Add a route `health` that responds with HTTP status code 200.

This task is of course child's play for a powerful model such as `gpt-4o`. OpenHands performs the following steps and only stumbles slightly:

1. The `app.py` file gets created. It contains the HTTP endpoint for `/health` and sets the web server port to 5000.
2. The `python3 /workspace/app.py` command starts the server as a background process.
3. The `curl http://localhost:5000/health` call fails with the error message, `Failed to connect to localhost port 5000`.
4. Due to the `lsof -i :5000` command, OpenHands attempts to check whether the server process is running on port 5000. This command also fails because `lsof` isn't installed in the sandbox container.
5. Because steps 3 and 4 were executed so quickly one after the other, there was no error message for the failed server start from step 2 in the log file. Only now does the error, `ModuleNotFoundError: No module named 'flask'` appear.
6. OpenHands fixes the problem with the installation of the Python module.
7. Calling `curl` again returns the expected result, and OpenHands returns control to the user.

After seven steps and a cost of \$0.05, the first task is completed. A human programmer would probably not have made the mistake with the missing module installation in the first place, and, if they had, would have noticed the problem immediately after the server start.

OpenHands will now extend the application to include simple user management with a database in SQLite format.

Prompt: Add code for a simple user administration, where users are stored in an SQLite database.

After several errors, all of which OpenHands can fix itself, there are two more routes to our backend. First, the list of all users can be called via GET. The second route is used to create new users by using POST. We've now reached 28 steps and a cost of \$0.29.

Next, we want to have a web frontend for the application:

Prompt: Add a web frontend where I can add a user to the database.

The input template on the very simple HTML page uses JavaScript without additional libraries, which is entirely to our liking. When submitting the form, the `fetch` function is used to send the entered data to the backend. OpenHands also extends the backend so that the website can be provided at the address, `http://localhost:5000/`.

However, we now also need a way to delete or edit the users.

Prompt: Add code for deleting and editing users.

OpenHands only takes care of the backend and adds the routes with HTTP methods DELETE and PUT. The DELETE call is then tested with a nonexistent user ID, which results in an error message. OpenHands apparently considers this a successful test and proudly reports that the code has been added and tested. In the meantime, 48 steps have been completed and we've spent \$0.59. The missing frontend code is added in just one step. As more tokens are required here, this step costs a little more.

Prompt: Add frontend code so that I can see a list of all users with Edit and Delete buttons.

Finally, we want to create a few sample users so that we can try out the application.

Prompt: Insert 5 example users.

OpenHands calls the REST API five times using `curl` and test user data, with each call costing \$0.02. This is good to know if you want to add 10,000 test users, for example.

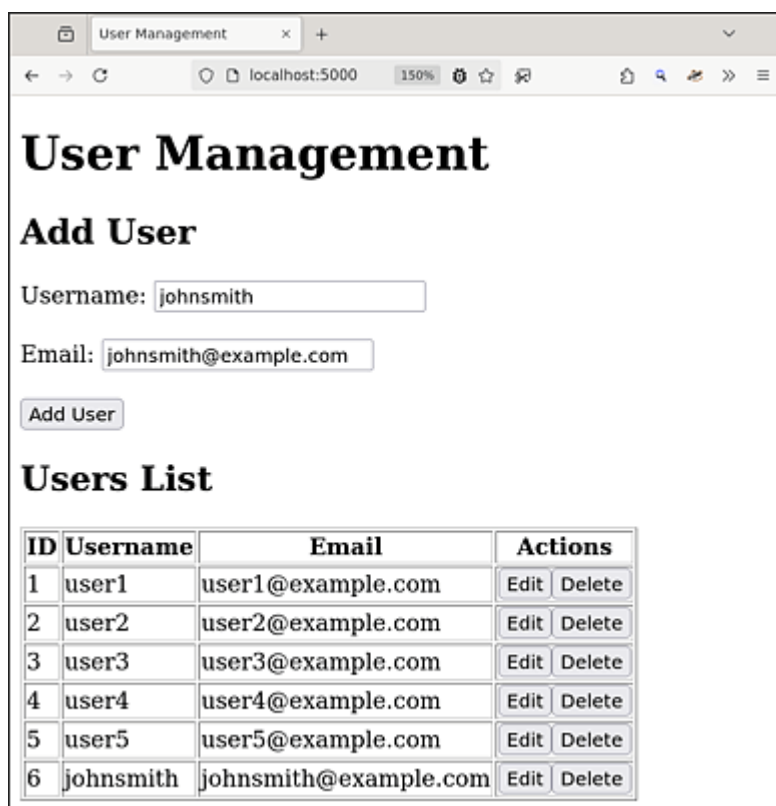


Figure 11.4 The “Finished” Web Application with a List of Users

Now you’ll rightly say that you can also use ChatGPT or another chatbot in the browser for this simple example. After all, you also receive precise instructions on which files and folders need to be created.

11.2.1 Not Too Much at Once!

Actually, we would have liked to present you with a complete example with a prompt in this form:

Prompt: Create a backend JSON REST API in Golang with a simple user administration. Users should have Email, Name, Password and get stored in an SQLite database. Add a frontend in `vue.js` where I can add, edit, and delete users.

The initial reaction from OpenHands is promising: a plan is presented in which the backend, including the database, is to be created first and then the frontend. But even the initialization of the Go module fails. If multiple attempts are made to resolve the problem, the folder for the Go module is created nested below the previous attempt. Unfortunately, the result was no better for the frontend code.

OpenHands stopped after 37 minutes and 60 steps without any significant result. Neither the desired backend in Go nor the frontend had any usable code. As this wasn't our first attempt that failed so miserably, we were at least able to use the time for a lunch break.

11.2.2 Conclusion

When we started working on the OpenHands sections and began to understand the technical concept behind it, our expectations were very high. The fascination with how well an LLM can deal with natural language and also with programming languages contributed to this. Disillusionment followed on its heels, however, and the failure of OpenHands due to trivial things, such as an incorrect path in the folder structure, leaves us with great doubts about the rapid implementation of an *autonomous software engineer*.

Nevertheless, OpenHands offers a very exciting outlook on how a part of software development could work in the future. This is still a very young project that will benefit massively from the further development of language models in particular.

We were most disappointed by the large discrepancy between local models and the OpenAI models. While local models for code completion or as a chat bot, achieve quite useful results, as we've presented in other chapters in this book, this was unfortunately not the case with OpenHands. If efficient software development is only possible in the future with access to

one of the tech giants (including the inevitable data monitoring), then that would be a very sad development for us.